

PDC Project · Spring 2026 · Team Coordination Plan

Team Members:

- **Sharique Baig (Team Leader)** — ERP: 28369
- **Suffiyan Asghar Ali** — ERP: 29182
- **Muhammad Usman** — ERP: 25177
- **Ashhal Aamir** — ERP: 29114

Big Picture:

We simulate **100 autonomous drones** in a decentralized swarm. No central brain — each drone senses local neighbors and acts on simple rules (Boids) that produce complex, emergent behavior.

Phases:

→ **2D (Python)** to prove logic → **3D Ursina** for rich visuals → **Gazebo (ROS2)** for real-robot-grade realism.

 Team Roles

Member	Role	Primary Domain
Suffiyan Asghar Ali	Physics & Environment Engine	World setup, simulation loop, collision physics, dynamic obstacles
Sharique Baig	Swarm Behavioral Logic	Boids rules, steering forces, task allocation, fault tolerance
Muhammad Usman	Visualization & UI/UX	Rendering (2D→Ursina→Gazebo), camera, dashboard, metrics display
Ashhal Aamir	Optimization & Parallelism	Spatial indexing, multiprocessing, performance logging, benchmarking

 Technology Progression

Phase 1 (2D)	Phase 2 (3D – Ursina)	Phase 3 (3D – Gazebo)
Python + Pygame NumPy kinematics Quadtree (2D) Multiprocessing CSV/JSON logs	→ Python + Ursina Engine 3D spheres / drone mesh Octree / Voxel Grid GPU-ready (VisPy optional) Real-time 3D dashboard	→ ROS2 + Gazebo Harmonic URDF drone models ROS2 topics / services PX4/MAVROS SITL ROS2 bag recording

 Standard Folder Structure

Agree on one folder structure first to ensure seamless integration:

pdc_project/	
├─ config.py	← Shared constants (Weights, radii, dt)
├─ environment.py	← Suffiyan (A1.1-A1.5): World & Obstacle logic
├─ swarm.py	← Sharique (B1.1-B1.5): SwarmManager physics
├─ optimization.py	← Ashhal (D1.1-D1.5): Spatial partitioning
├─ visualizer.py	← Usman (C1.1-C1.5): Rendering loop
└─ main.py	← Shared entry point

Shared Data Contracts

- 1. **drone_positions**: `np.ndarray(shape=(100, 2))` — Sharique provides this (B1.1)
- 2. **drone_velocities**: `np.ndarray(shape=(100, 2))` — Sharique provides this (B1.2)
- 3. **neighbor_map**: `dict[int, list[int]]` — Ashhal provides this (D1.2)
- 4. **obstacles**: `list[pygame.Rect]` — Suffiyan provides this (A1.2)

🚩 MILESTONE 1 — Environmental Modeling and Drone Behavior Foundation

Goal: Establish the simulated environment and foundational drone control logic in 2D.
Duration: Weeks 1–3 | **Trigger to proceed:** 100 drones spawn randomly, move, and sense neighbors without collisions.

Suffiyan — Physics & Environment Engine

Task ID	Task	Details	How-To
A1.1	Simulated Environment	1000×1000 unit continuous plane. Define boundary walls with configurable: wrap-around (toroidal) OR hard-wall repulsion.	<code>pygame.display.set_mode((1000,1000))</code> + wall constants
A1.2	Static Obstacle Placement	Place 5–10 rectangular obstacles at fixed positions. Store as list of <code>Rect(x,y,w,h)</code> .	<code>pygame.draw.rect()</code> per obstacle
A1.3	Main Simulation Loop	<code>while running:</code> <code>handle_events() →</code> <code>update_physics() → render().</code> Fixed time step $\Delta t = 0.02s$.	Python <code>time.perf_counter()</code> for tick timing
A1.4	Boundary Handling	When drone within <code>d_wall=20</code> of boundary, apply outward repulsion force $F = k / dist^2$.	Override velocity component on boundary hit
A1.5	Environment Config	Create <code>config.yaml</code> — world size, num_drones, dt, seed, obstacle list.	PyYAML or json

Sharique — Swarm Behavioral Logic

Task ID	Task	Details	How-To
B1.1	Drone Representation	Properties: <code>id</code> , <code>pos [x,y]</code> , <code>vel [vx,vy]</code> , <code>heading θ</code> . Methods: <code>update_position(dt)</code> .	NumPy arrays for pos/vel
B1.2	2D Kinematics	<code>pos += vel * dt</code> . Clamp scaled speed (<code>v_max=250</code> for <code>dt=0.02</code>).	NumPy vector ops
B1.3	Random Initialization	Scatter 100 drones randomly in world. Use <code>np.random.seed(42)</code> .	<code>np.random.rand()</code>
B1.4	Basic Drone Sensing	Each drone detects neighbors within a fixed interaction radius <code>R</code> .	Share global positions array
B1.5	Reactive Avoidance	Balance <code>safety_distance</code> (20) forces to form non-colliding emergent lattice structures.	Strong separation override

Usman — Visualization & UI/UX

Task ID	Task	Details	How-To
C1.1	Foundation Rendering	Create 800×600 window. Clear → draw obstacles → draw drones → flip.	<code>pygame.init()</code> , <code>clock.tick(60)</code>
C1.2	Drone Rendering	Each drone = small circle (radius 4px). Visualize individual drone motion.	<code>pygame.draw.circle(surf, color, pos, 4)</code>
C1.3	Neighbor Awareness Viz	Draw lines or highlights to visualize neighbor awareness for local sensing.	<code>pygame.draw.line()</code> for detected neighbors
C1.4	FPS & Metric Overlay	Top-left corner: real-time FPS display and drone count.	<code>pygame.font.render()</code>
C1.5	Interactive Controls	Press <code>SPACE</code> to pause/resume. Basic camera zoom/pan in 2D.	Boolean <code>paused</code> flag + view offset

Asshal — Optimization & Parallelism

Task ID	Task	Details	How-To
D1.1	Naïve Neighbor Check	For each drone, check all others. $O(N^2)$ baseline for benchmarking.	Double nested loop over drone list
D1.2	Spatial Partitioning	Implement Grid Hash to reduce neighbor-check complexity ($O(N)$ expected).	Dict keyed by (<code>cell_x</code> , <code>cell_y</code>)

Task ID	Task	Details	How-To
D1.3	Quadtree Optimization	Implement Quadtree as an alternative spatial indexing method.	<code>QuadTree</code> class or <code>scipy.spatial.KDTree</code>
D1.4	Performance Logging	Log FPS and CPU usage every 100 ticks to <code>benchmark_log.csv</code> .	Python <code>csv</code> module
D1.5	Data Parallelism Ready	Vectorize state updates using NumPy to prepare for parallel execution.	<code>np.ndarray</code> broadcasting

MILESTONE 2 — Decentralized Coordination Algorithms

Goal: Implement distributed algorithms enabling collective behavior without central supervision.

Duration: Weeks 4–6 | **Trigger to proceed:** Emergent coordinated motion (flocking/formation) visible.

Suffiyan — Physics & Environment Engine

Task ID	Task	Details	How-To
A2.1	Collision Enhancements	Integrate avoidance with alignment and task decisions.	Weighted steering forces
A2.2	Predictive Avoidance	Project positions forward and adjust if future collision is detected.	<code>pos_pred = pos + vel * k * dt</code>
A2.3	Dynamic Environment	Add 1–2 moving rectangular obstacles for drones to avoid.	<code>obs_pos += obs_vel * dt</code>
A2.4	Physics Tuning	Fine-tune repulsion constants and safety zones for dense swarms.	Config hot-reload
A2.5	Obstacle Sensing	Enhance local sensing to include nearby obstacle edges.	<code>F_obs</code> repulsive force

Sharique — Swarm Behavioral Logic

Task ID	Task	Details	How-To
B2.1	Reynolds Flocking	Implement Alignment, Cohesion, and Separation rules for group movement.	Weighted sums of steering vectors
B2.2	Decentralized Task Allocation	Assign sub-tasks (waypoints) based on proximity or priority.	Auction protocol via local communication
B2.3	Consensus Mechanisms	Drones agree on movement direction or task priorities without global supervision.	Lightweight message-passing/priority vote

Task ID	Task	Details	How-To
B2.4	local communication protocols	Each drone broadcasts intent (task, heading) to neighbors.	Local broadcast within radius R
B2.5	Formation Control	Define and maintain shapes (V-formation, grid) via target-position injection.	Shared intent formation

Usman — Visualization & UI/UX

Task ID	Task	Details	How-To
C2.1	Interactive UI Dashboard	Add sliders to tweak weights: Separation, Cohesion, Alignment in real-time.	<code>pygame_gui</code> or custom sliders
C2.2	Formation Visualization	Highlight formation centroids and target positions.	Virtual leader rendering
C2.3	Task Status HUD	Visualize task zones and their assignment status (Unclaimed, Assigned, Done).	Color-coded regions
C2.4	Debug Force Vectors	Draw lines showing current steering forces for each rule.	Scaled vector lines from drone center
C2.5	Collision Counters	Real-time display of total collision count for validation.	Global counter in simulation

Asshal — Optimization & Parallelism

Task ID	Task	Details	How-To
D2.1	Multiprocessing Updates	Distribute drone updates across multiple CPU cores using <code>multiprocessing.Pool</code> .	<code>pool.starmap()</code> for chunk updates
D2.2	Shared Memory States	Use <code>SharedMemory</code> to avoid pickling overhead between processes.	<code>multiprocessing.shared_memory</code>
D2.3	Bottleneck Analysis	Use <code>cProfile</code> to identify slow behavioral logic components.	<code>python -m cProfile</code>
D2.4	Parallel Performance Log	Compare Sequential vs Parallel FPS on 1, 2, 4 cores.	<code>M2_benchmarks.csv</code>
D2.5	Communication Overhead	Measure and log total messages sent per tick per drone to ensure scalability.	In-node counters

 MILESTONE 3 — Task Execution and Team Synergy Validation

Goal: Integrate all components to demonstrate emergent teamwork in a multi-phase mission.

Duration: Weeks 7–9 | **Final Deliverable:** 100 drones performing coordinated tasks in 3D.

Suffiyan — Physics & Environment Engine

Task ID	Task	Details	How-To
A3.1	Dynamic 3D Obstacles	Add moving spheres/cubes specifically for the 3D environment.	<code>obs3d.pos += sin(t)</code>
A3.2	Area Coverage Scenario	Implement a mission where drones must cover a 3D volume efficiently.	Voxel-grid visit tracking
A3.3	Fault Injection	Randomly "fail" (remove) 10-25% of drones to test swarm robustness.	Random deletion from drone list
A3.4	Gazebo World Design	Create the final 3D world in SDF format with obstacles and boundaries.	Gazebo <code>.world</code> XML
A3.5	URDF Integration	Define the drone's collision and inertial properties for the physics engine.	<code>quadrotor.urdf</code>

Sharique — Swarm Behavioral Logic

Task ID	Task	Details	How-To
B3.1	Mission Execution Logic	Coordinate behavior based on multi-phase mission objectives (e.g., target tracking).	High-level mission state machine
B3.2	Real-time Adaptations	Implement behavioral adjustments if drones fail or obstacles move dynamically.	Dynamic re-bidding for tasks
B3.3	Team Synergy Validation	Ensure robust behavior under disruptions and drone failures.	Test recovery time metrics
B3.4	ROS2 Coordination Node	Bridge the swarm logic with Gazebo via ROS2 topics (Odom, CmdVel).	<code>rclpy</code> publishers/subscribers
B3.5	Formation Traversal	Move a 100-drone formation through an obstacle field in 3D.	Coordinated waypoint following

Usman — Visualization & UI/UX

Task ID	Task	Details	How-To
---------	------	---------	--------

Task ID	Task	Details	How-To
C3.1	3D Simulation (Ursina)	Switch to Ursina Engine for high-fidelity 3D visualization.	<code>from ursina import *</code>
C3.2	3D Drone Rendering	Replace circles with 3D Sphere/Drone models. Scale and rotate based on vel.	<code>Entity(model='sphere')</code>
C3.3	Camera Navigation	Implement Orbit, Pan, and Zoom controls for the 3D scene.	<code>EditorCamera()</code>
C3.4	Performance Visualization	Render heatmaps showing area coverage density in real-time.	3D grid with transparency
C3.5	Final Demo Recording	Use Gazebo/RViz recording tools to capture mission execution.	<code>rosviz</code> + screen capture

Asshal — Optimization & Parallelism

Task ID	Task	Details	How-To
D3.1	3D Octree	Extend spatial partitioning to 3D for efficient neighbor discovery.	<code>Octree</code> with 3D distance check
D3.2	GPU Acceleration Ready	(Optional) Port critical compute paths to GPU via PyCUDA/CuPy.	Vectorized GPU kernels
D3.3	Final Performance suite	Metrics: mission completion time, collision count, area coverage, robustness.	Quantitative analysis script
D3.4	Scalability Comparison	Comparative analysis: impact of swarm size (25, 50, 100) on performance.	Multi-run benchmark
D3.5	Final Logging Suite	Export all performance metrics for the project report.	CSV to Matplotlib plots

 Evaluation Metrics

Metric	Target	Description
Mission completion time	Minimize	Ticks to complete all assigned tasks
Collision count	0 (ideal)	Total inter-drone and drone-obstacle collisions
Area coverage %	>95%	Fraction of target area covered within time limit
Robustness	>90% at 25% failure	Coverage maintained after drone removal

 Timeline

Week	Focus	Milestone
1-2	Environment + drone model + spatial indexing	M1
3	Collision avoidance + foundational visualization	M1
4-5	Reynolds flocking + decentralized allocation	M2
6	Integration testing + parameter tuning	M2
7-8	Mission scenarios + fault injection + Ursina Port	M3
9	Performance evaluation + Gazebo Demonstration	M3

Prepared for PDC Course Project, Spring 2026